BackyardPay

BP

Shared records for backyard rentals

Mobile rental management and payment records for landlords and tenants in East London, Eastern Cape.

STUDENT	MODULE
Mbongiseni Nkambule Student number: 231064977	Application Technology IV APT41P2
PROJECT	SUBMISSION
Individual project Application release: 1.4.0 (build 13)	Project portfolio 12 November 2026

Project links

Open the GitHub source repository

https://github.com/nkambuleinnocent52-Ghost/Mbongiseni-Nkambule-BackyardPay

Open the deployed BackyardPay application

https://quiet-pastelito-19fa50.netlify.app

Document guide

Context: page 2 · Goal and objectives: page 3 · User manual: pages 4–8 · Code appendix: pages 9–12 · References: page 13.

Prepared against the supplied 2026 project guideline. Functionality is documented through screenshots and steps; the class message specifies a seven-minute presentation without a live demo.

1. Context and background

The problem

Backyard rental arrangements can involve cash, bank transfers and messages kept in different places. When a landlord relies on a notebook or scattered messages, it can be difficult to identify who has paid, which month a payment covers and what remains outstanding. A tenant may also struggle to retrieve an agreed payment record when a disagreement arises. A payment reported by a tenant can be mistaken for money the landlord has actually received.

BackyardPay addresses this record-keeping problem by linking properties, assigned rooms, tenants and payment records in one system. Landlords and tenants see the same rental relationship through separate, authorised views. The intended benefit is clearer records and easier follow-up; the project does not claim measured reductions in disputes or results from an unperformed local field study.

Target community and users

The intended setting is small backyard rental properties in East London, Eastern Cape. The landlord manages rooms, rent and deposits. The tenant checks their own rental information, reports payments already made and reports problems. A tenant account connects to a room already assigned by the landlord through a private invitation. The application is not a property-search marketplace.

Why mobile access is suitable

A tenant can record a cash or bank payment from a phone, view their balance and retrieve a receipt without returning to a desktop computer. A landlord can check records at a property, acknowledge a leaking tap report and capture the property location with permission. A desktop interface can support the same accounting tasks, but normally ties use to a computer. BackyardPay adapts navigation and layout to smaller screens as well as larger browsers (Flutter, n.d.).

Scope and boundaries

The current implementation is a Flutter/Dart application deployed as a responsive website on Netlify, with Supabase authentication and PostgreSQL storage. It records cash, EFT and other payment methods but does not transfer money or verify a bank account. Only landlord-confirmed rent payments reduce recorded rent balances. Deposits remain separate from rent.

Rent is normally due on the 1st. A first partial month can be calculated from actual calendar days, including move-in day. The ten-day deposit deadline is the project’s agreed product policy, not a claim that South African law gives every tenant a universal ten-day grace period. Deposit deductions must be supported and do not automatically forfeit the entire deposit (Republic of South Africa, 1999).

2. Goal and objectives

Goal: Provide landlords and tenants with a simple, shared and traceable record of backyard rentals, rent payments, security deposits and reported problems.

Objectives: (1) Register users and link tenants securely to assigned rooms. (2) Manage properties, room availability and room-specific rent. (3) Calculate first-month rent and track monthly balances. (4) Report and confirm rent/deposit payments separately, with receipts. (5) Support issue acknowledgement and in-app notifications.

INPUT	PROCESS / SCREEN	OUTPUT
Property, room, monthly rent, deposit and move-in date	Property setup and tenant onboarding; calculate first-month charge	Available rooms and itemised move-in costs
Payment amount, date, method and rent month	Tenant report; landlord receipt check and confirmation	Pending/confirmed status, updated balance and receipt
Issue location and description	Reported problems; landlord acknowledgement and reply	Shared issue status and in-app notification

Success is assessed through working task flows, calculation/access checks and responsive layouts. Automated checks support functionality; they do not replace observation of ordinary users.

3.1 Access and role dashboards

Open the application link in a browser. Choose Get started, select Landlord or Tenant, then log in with your own email and password. A new landlord creates an account. A new tenant uses the private invitation supplied by their landlord; they do not choose a room during registration.

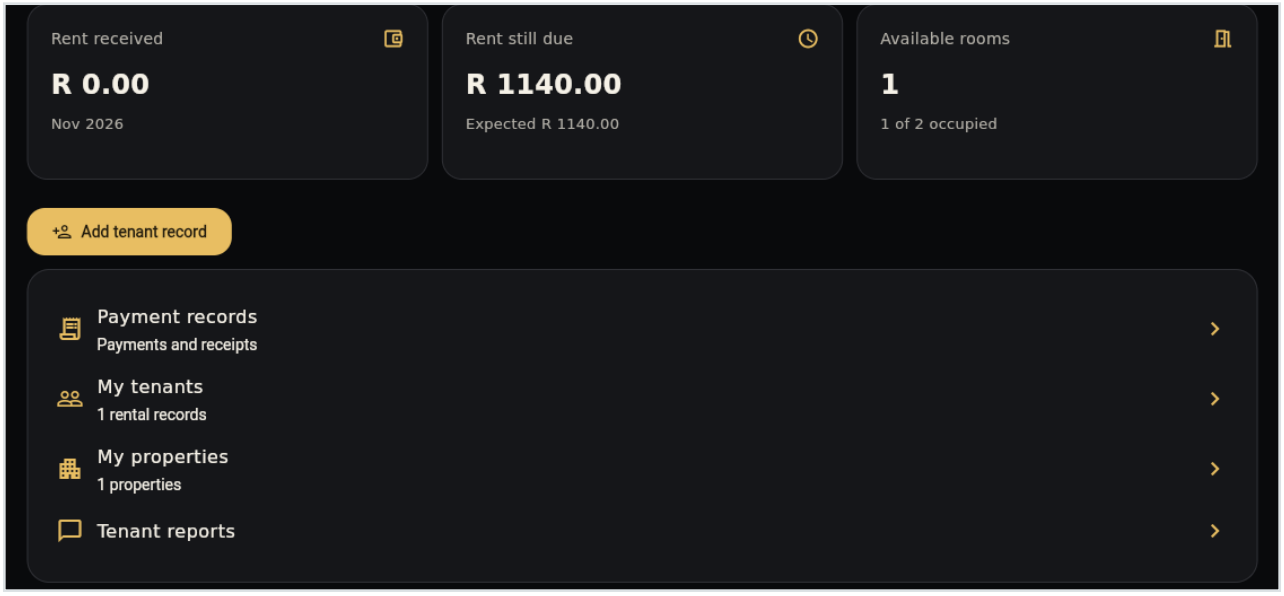
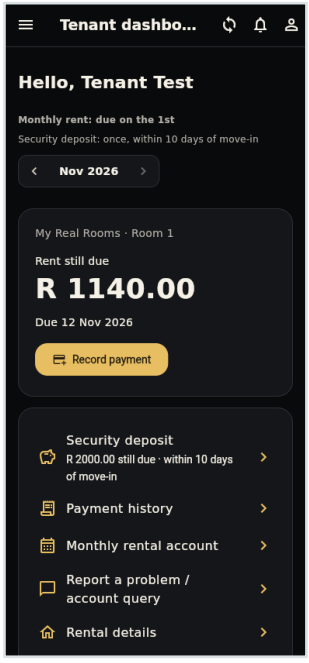


Figure 1. Landlord home: confirmed rent, rent still due, rooms and the main management links.

1. Choose the month. 2. Read Rent received, Rent still due and Available rooms. 3. Use My properties, My tenants or Payment records. Pending payments appear for checking when present.



4. Tenant view. Check the assigned property/room, selected month and rent still due. A zero balance means no remaining rent for that selected month; check the monthly account for other months.

5. Select Record payment to report money already paid. Security deposit, Payment history, Monthly rental account and Rental details provide the supporting records.

6. Use the phone menu for navigation. The bell opens Notifications; Account provides Edit profile, Change password and Logout.

Figure 2. Tenant home on a phone.

Screenshot note: the illustrated manual uses release 1.4.0 and isolated demonstration accounts. Values are examples, not real transactions; dates and amounts vary between task captures. Detail crops improve readability. Test clocks were advanced for deadline behaviour.

3.2 Properties, rooms and onboarding

1. Landlord: open My properties → Add property. Enter the address, room count, default monthly rent and separate security deposit; save. Under Room rent, select larger rooms and save a custom monthly amount. Other rooms retain the default for new tenancies.

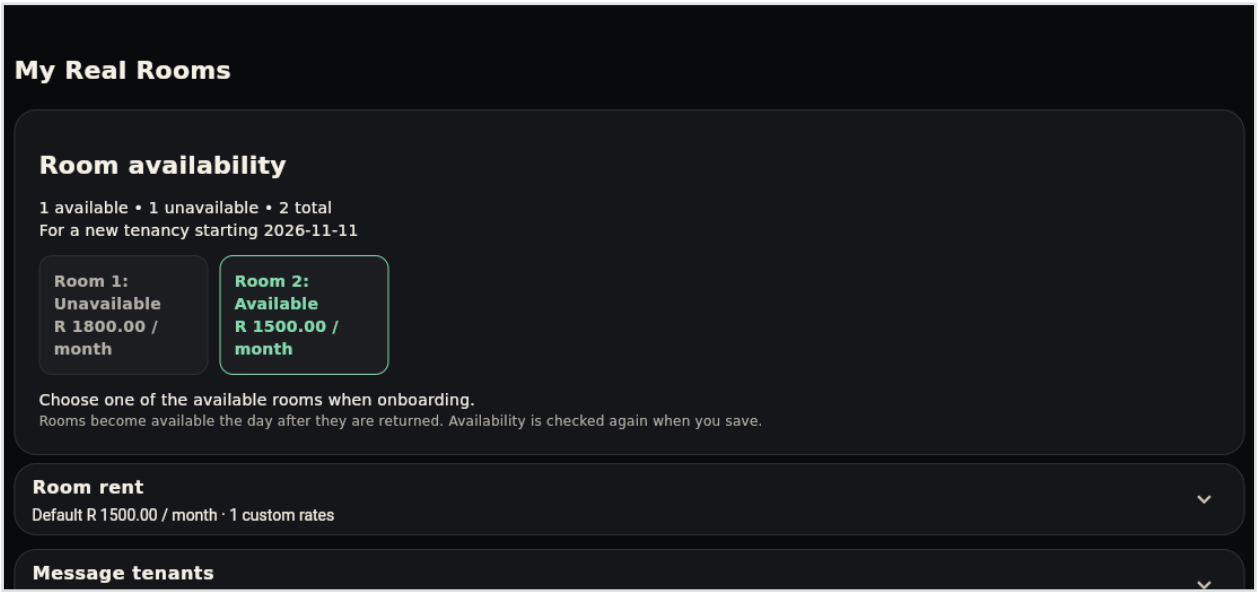


Figure 3. Property room availability: the landlord assigns an available room.

2. Choose Add tenant record, select the property and an available room, and enter the tenant's name/contact details and actual move-in date. Check the separate first-rent and deposit amounts before saving.

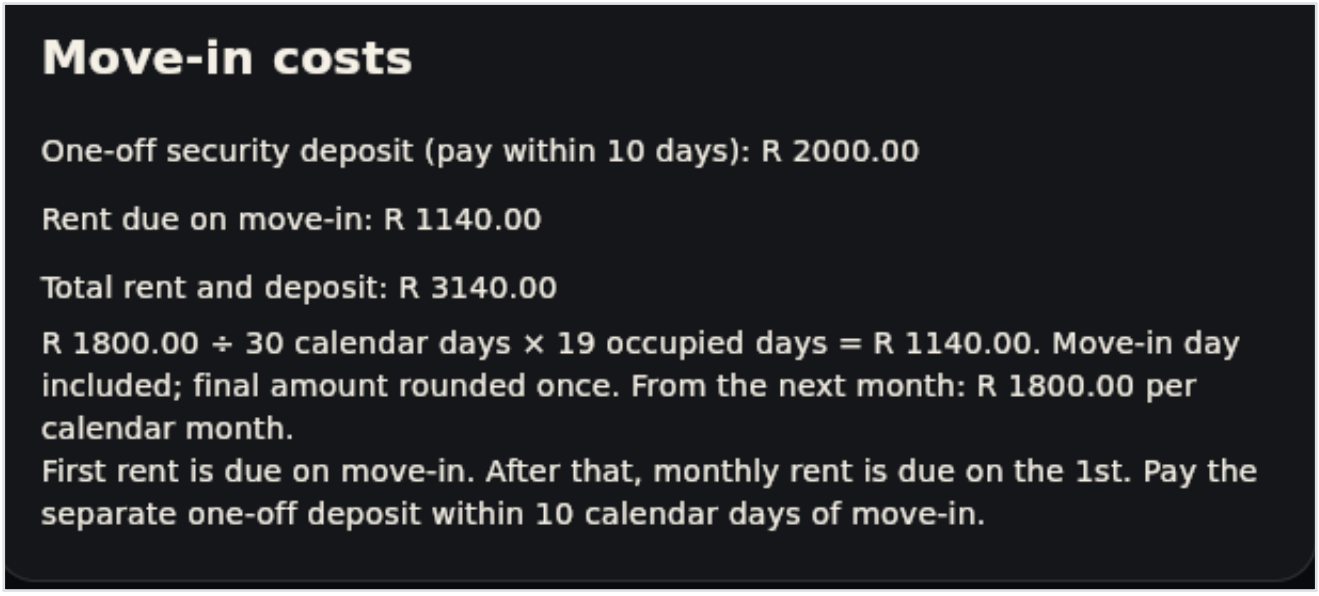


Figure 4. Move-in cost preview: first rent and the one-off deposit are itemised before saving.

3. Open the rental's Invite this tenant section, copy the private JOIN invitation and share it privately. The tenant creates their own login using that invitation. Existing rental history remains linked; a public property code cannot assign a room.

4. To record GPS coordinates, open the property and choose Capture current location while at the property. Allow browser permission. Location capture needs HTTPS; entering a street address does not itself capture GPS coordinates.

3.3 Record rent and confirm receipt

1. Pay the landlord through the usual cash or banking arrangement. In the tenant account, select Record payment and keep Payment for set to Rent. Enter the amount, payment method, date and the rent month. Bank methods require the payment reference.

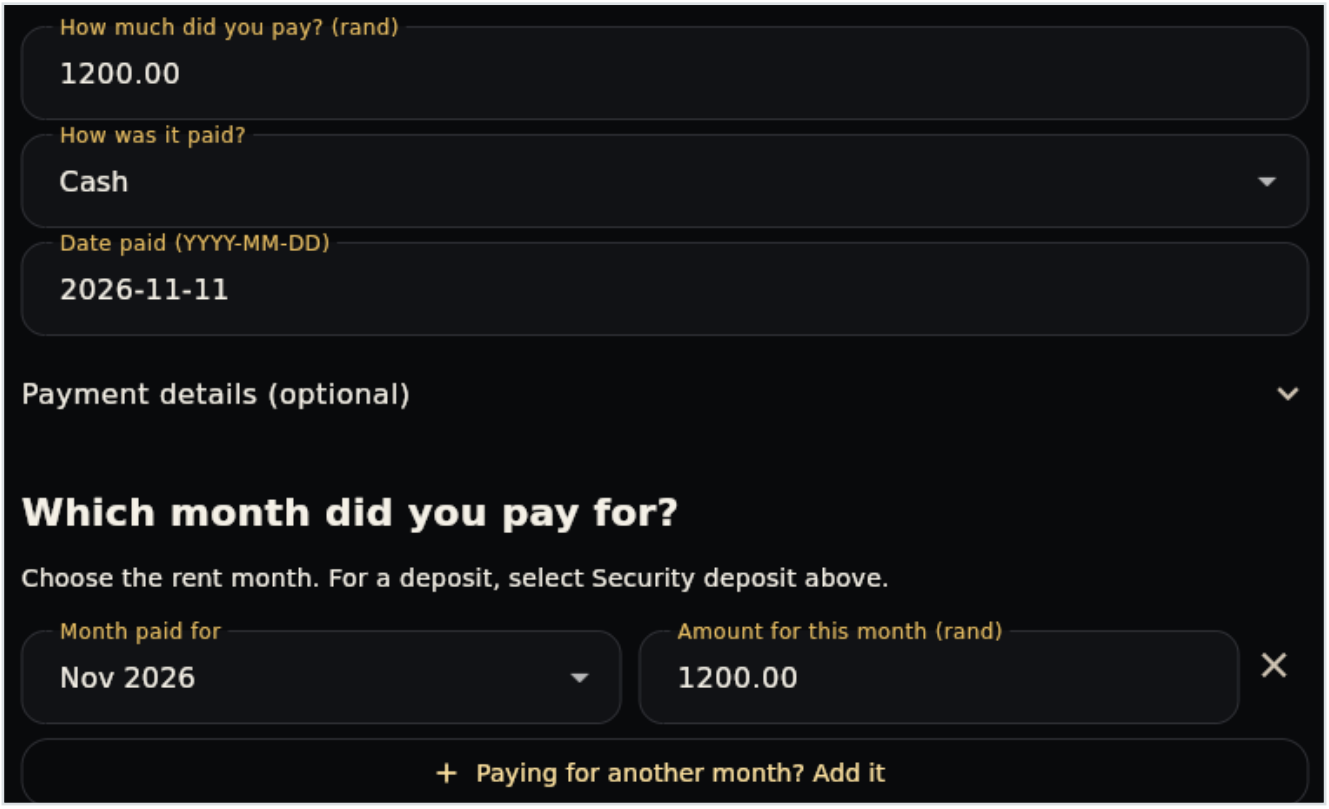


Figure 5. Rent-entry detail: amount, method, date and rent month; advance months are available.

2. Check the month amount; use Paying for another month? Add it only when the same payment covers several months. Optional payment details let you name another payer. Submit payment. The report remains Pending, so it does not yet reduce confirmed rent owed.

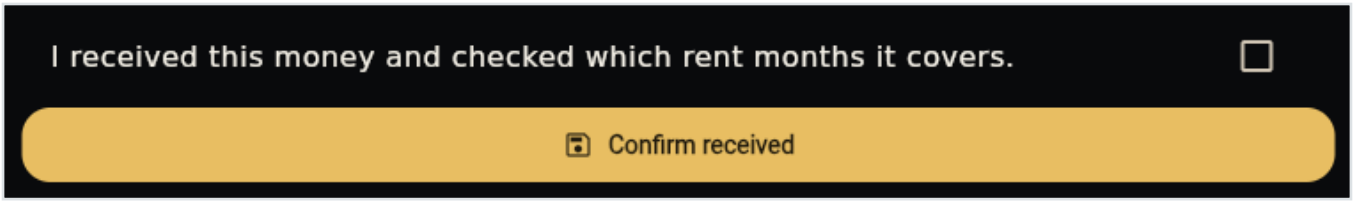


Figure 6. Landlord checking the reported payment before confirming money received.

3. Landlord: open the pending payment from the dashboard or Payment records. Check actual cash received or your bank record. Check the amount and months, tick the receipt confirmation and choose Confirm received. A mistaken report can instead be rejected with an explanation. Never confirm only because a tenant submitted a report.

3.4 Receipts, history and deposit

1. Open Payment history / Payment records. Select the month and All, Pending, Confirmed or Rejected. Open a confirmed payment to see the months covered and its receipt. Use Download / share receipt or Print / save PDF.

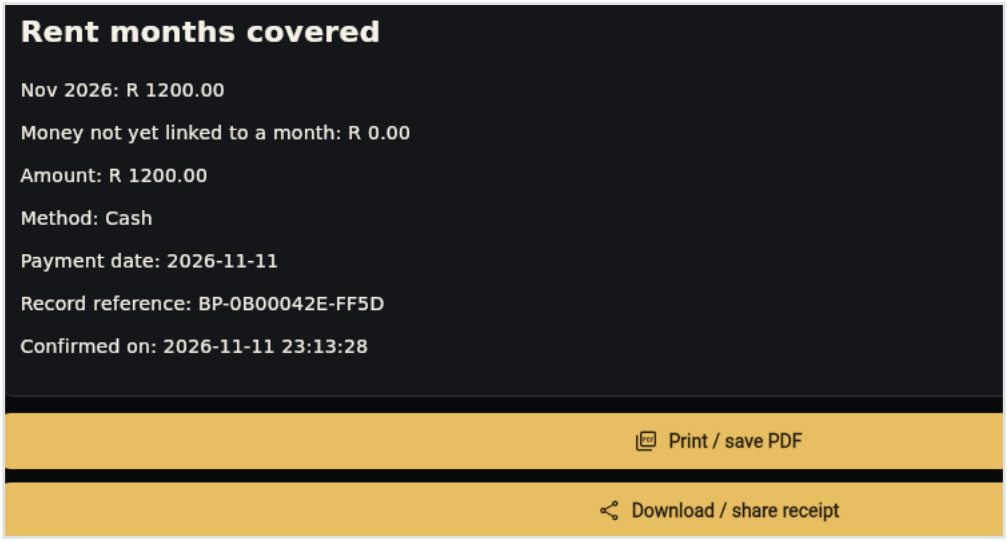


Figure 7. Receipt detail: rent month, amount, method, date and download/share controls.

2. Open Monthly rental account to review charges and payments across months, earlier unpaid rent and credit not yet assigned to a month. A payment for December should cover December; it must not erase an unrelated November balance.

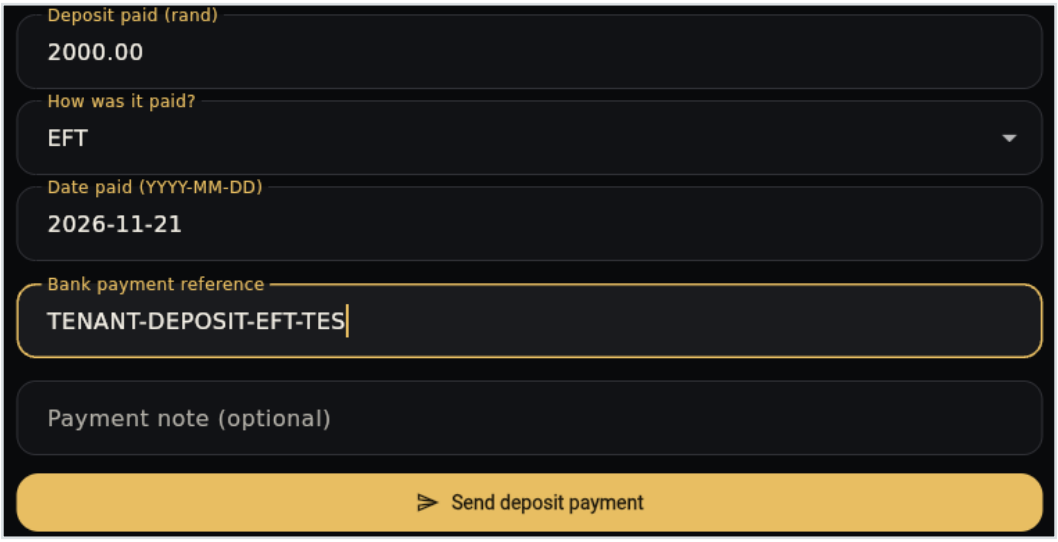


Figure 8. A separate tenant deposit report; it is not treated as monthly rent.

3. To report a deposit, choose Record payment → Payment for → Security deposit (or the deposit link in Rental details). Enter the actual amount/date/method and send it. The landlord checks and confirms the deposit separately. New-tenancy deposits are due once within ten days of move-in; unpaid amounts trigger in-app reminders.

4. A pending deposit report waits for review, rather than asking the tenant to pay that reported amount again. At move-out, the landlord records a return or a supported deduction and its reason. Damage does not automatically remove the entire deposit.

3.5 Problems, notices and move-out

1. Tenant: select Report a problem / account query. Enter where the problem is and describe it, then Send issue. Report matters such as a shared leaking tap, broken gate or a missing payment record. A report does not automatically decide who must pay for damage.

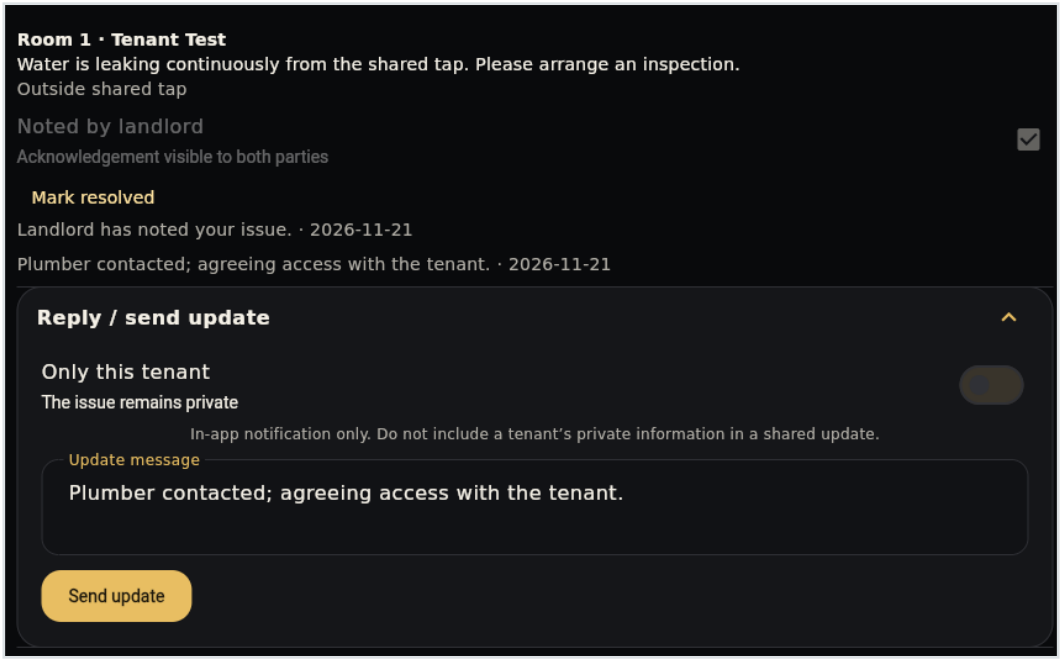


Figure 9. Landlord acknowledgement and follow-up: the tick and updates are visible to both parties.

2. Landlord: open Tenant reports and tick Noted by landlord to acknowledge receipt. Use Reply / send update. Keep an individual issue private; choose a property-wide update only for information relevant to everyone. Mark resolved when the issue has been addressed.



Figure 10. Deposit deadline warning in the tenant dashboard; report payment only if already paid.

3. Open the bell / Notifications to read messages and mark them as read. Enable browser alerts only if wanted. Notices remain in the account; browser alerts operate while the app is open. These are not SMS or remote push messages.

4. Landlord: use Rental details → Monthly rent for an agreed future rent change; dated changes stay listed. Under Move-out, record actual room/key return. The room becomes available for a new tenancy from the following day; unpaid balances and deposit records are retained.

Mobile features: responsive phone navigation, permission-based geolocation, in-app notices, optional active-browser alerts and PDF download/share. This submission documents a responsive web implementation; native Android delivery and background push have not been demonstrated.

A1. Structure and role navigation

The source is organised by feature and role. main.dart initialises Supabase; app.dart handles sessions and application routing. Screens present tasks, data/ holds state and accounting, models/ defines records and services/ handles receipts, alerts and authentication callbacks.

```
lib/
  main.dart / app.dart
  screens/
    auth/          login, registration, password reset
    landlord/      landlord home and tenant list
    tenant/        tenant home
    shared/        navigation, profile and common UI
    properties/    property setup, room rent, availability
    payments/      rent/deposit reports and confirmation
    rentals/       rental details, reports, deposit, move-out
  data/            store, cloud gateway, ledger calculations
  models/          account, property, tenancy, payment
  services/        receipts, alerts and auth callbacks
  ui/              theme and reusable visual components
supabase/migrations/   server rules and row-level security
test/ and tool/        automated checks and release tools
```

A1.1 Select the signed-in role

Source: lib/screens/shared/dashboard.dart, lines 14–26.

```
14  class DashboardScreen extends StatelessWidget {
15    const DashboardScreen({super.key});
16    @override
17    Widget build(BuildContext context) {
18      final user = StoreScope.of(context).currentUser;
19      if (user == null) return const Scaffold(body: SizedBox.shrink());
20      return user.role == Role.landlord
21          ? const LandlordDashboardScreen()
22          : const TenantDashboardScreen();
23    }
24  }
```

StoreScope gives the widget the current account. A missing account renders an empty scaffold; otherwise the enum selects the correct dashboard. Each role page is an independent Dart library. UI routing improves usability, but it is not the security boundary: database checks must also restrict access (Supabase, n.d.).

A1.2 State and interaction

StoreScope is an InheritedNotifier for AppStore. A saved action refreshes the shared snapshot, allowing related screens to display updated records. The same navigation and visual components are reused across role pages. Short task labels, a visible pending state and optional detail sections apply consistency, feedback and reduced memory-load principles from the supplied HCI slides (APT41P2 teaching materials, n.d.).

A2. Financial calculations and rooms

A2.1 Calendar-day first rent

Source: lib/data/research_ledger.dart, lines 69–75.

```
69  /// Calendar-day billing, including the move-in day. Round the final cents only.
70  int daysInMonth(DateTime date) => DateTime(date.year, date.month + 1, 0).day;
71  int proratedFirstCharge(int monthlyCents, DateTime start) {
72    final days = daysInMonth(start), occupied = days - start.day + 1;
73    return (monthlyCents * occupied * 2 + days) ~/ (2 * days);
74  }
75
```

Amounts are integer cents, avoiding repeated floating-point money rounding. DateTime(year, month + 1, 0) gives the last day of the month. occupied includes the move-in date. Integer arithmetic rounds the final charge once, rather than rounding a daily rate first.

WORKED EXAMPLE	RESULT
Monthly rent R1,500; January has 31 days; move-in 15 January	31 − 15 + 1 = 17 occupied days
R1,500 × 17 ÷ 31	First rent R822.58
Separate deposit R2,000	Total initial charges R2,822.58; deposit deadline tracked separately
From the following full month	R1,500 rent due on the 1st

A2.2 Prevent assigning an occupied room

Source: lib/data/room_availability.dart, lines 11–28.

```
11    final day = DateTime(moveIn.year, moveIn.month, moveIn.day);
12    final blocked = <int>{};
13    for (final t in tenancies.where((t) => t.propertyId == property.id)) {
14      final end = actualHandover ? t.endedOn : t.leaseEnd;
15      if (end == null || !DateTime(end.year, end.month, end.day).isBefore(day)) {
16        final room = int.tryParse(t.room);
17        if (room != null) blocked.add(room);
18      }
19    }
20    return [
21      for (var room = 1; room <= property.rooms; room++)
22        if (!blocked.contains(room)) room,
23    ];
24  }
```

The helper filters tenancies for the selected property and builds a set of blocked room numbers. Actual room return, not a planned departure date, determines availability. The save operation also checks availability on the server, protecting against two sessions selecting the same room. Room-specific rent is copied into new tenancy terms; changing a default does not silently rewrite an existing tenant’s agreed rent.

A3. Payment validation and storage

A3.1 Validate month amounts and receipt confirmation

Source: lib/screens/payments/payment_entry.dart, lines 104–125.

```
104        final seen = <String>{};
105        var sum = 0;
106        for (var i = 0; i < periods.length; i++) {
107          final cents = centsOrZero(allocated[i].text);
108          if (cents == 0) continue;
109          if (!seen.add(periods[i]))
110            throw ArgumentError(
111              'You selected the same month twice. Enter its total in one row.',
112            );
113          splits.add({'period': periods[i], 'amountCents': cents});
114          sum += cents;
115        }
116        final p = widget.paymentId == null ? null : s.payment(widget.paymentId!);
117        final limit = widget.credit ? p!.creditCents : parseMoney(amount.text);
118        if (sum > limit)
119          throw ArgumentError(
120            'The amounts entered for the months are more than the payment. Reduce them.',
121          );
122        if (owner && !received)
123          throw ArgumentError(
124            'Tick the box to confirm that you received the money and checked which months it
     covers.',
125          );
```

Each non-zero month amount becomes one allocation. Duplicate months are rejected. The sum cannot exceed the actual payment amount. A landlord must explicitly confirm receipt; the tenant does not get this checkbox. These checks prevent a payment from being counted twice or a report from being mistaken for verified funds.

A3.2 Send the operation and refresh shared records

Source: lib/data/cloud_gateway.dart, lines 29–42.

```
29      final response = Map<String, dynamic>.from(
30        await client.rpc(
31          'bp_action',
32          params: {'operation': operation, 'args': args},
33        ),
34      );
35      try {
36        response['snapshot'] = await snapshot();
37      } catch (_) {
38        throw StateError(
39          'Your change was saved, but updated records could not be loaded. Refresh before
    trying again.',
40        );
41      }
42      return response;
```

CloudGateway calls bp_action through Supabase RPC and refreshes the snapshot after success. If saving succeeds but refresh fails, the user is told to refresh before retrying. Supabase Auth handles authentication; passwords are not stored in rental records (Supabase, n.d.).

A4. Deposit follow-up and access

A4.1 Due date, pending reports and reminders

Source: lib/data/deposit_reminders.dart, lines 63–88.

```
63    final customDue = explicitLegacyDate
64        ? _recordDate(tenancy.record['depositDueOn'])
65        : null;
66    final dueOn = customDue ?? actualStart?.add(const Duration(days: 10));
67    if (dueOn == null) return null;
68
69    final date = _calendarDate(today);
70    final eligibleOn = dueOn;
71    final daysSinceDue = date.difference(dueOn).inDays;
72    final outstanding = ledger.depositOutstanding(tenancy)!;
73    final pending = reports
74        .where((r) => r['tenancyId'] == tenancy.id && r['state'] == 'pending')
75        .fold<int>(0, (sum, r) {
76          final amount = r['amountCents'];
77          return sum + (amount is num ? math.max(0, amount.toInt()) : 0);
78        });
79
80    return DepositReminderInfo(
81      dueOn: dueOn,
82      eligibleOn: eligibleOn,
83      daysSinceDue: daysSinceDue,
84      outstandingCents: outstanding,
85      pendingCents: pending,
86      unreportedCents: math.max(0, outstanding - pending),
87      overdue: outstanding > 0 && !date.isBefore(eligibleOn),
88    );
```

For a new tenancy, the deadline is actual move-in plus ten calendar days; explicit legacy agreements are preserved by the earlier branch. The warning becomes eligible on the deadline itself. Pending reports are not confirmed receipts, but their value is subtracted from the amount a tenant is asked to report/pay again. Ended or zero-deposit tenancies are excluded earlier in the function.

A4.2 Server permissions

PostgreSQL row-level security is enabled for rental tables. A tenant can read their linked tenancy; the owning landlord can read and manage their property’s records. The notices policy restricts reads to the recipient. Server functions also check permission before mutation, so hiding a screen is not the only protection (Supabase, n.d.).

A4.3 Recipient-only notice policy

Source: supabase/migrations/202610240001_backyardpay.sql, lines 73–73.

```
73  create policy bp_notice_read on public.bp_notices for select to authenticated
    using(user_id=auth.uid());
```

A4.4 Verification and limits

Release 1.4.0 passed 84 Flutter tests and static analysis across 67 Dart files with zero issues. Additional navigation/session checks passed. A Chromium journey with a simulated Supabase transport covered assigned-room signup, room rates, rent confirmation, advance rent, receipts, deposit warnings/confirmation, issues and account recovery. These checks do not prove live email delivery, native-device behaviour, user satisfaction or production load capacity (Nkambule, 2026).

References

1. Application Technology IV teaching materials (n.d.) Supplied HCI lecture slides: Chapters 1–3 (Part 1), 5–7, 9 (Part 1), 12 and 13. Learning, memory, interaction, navigation, usability rules, evaluation and stakeholder context. Chapter 12’s internal title refers to Chapter 21: Hypertext, multimedia and the WWW.

2. Department of Business and Application Development (2026) Application Technology IV (APT41P2): Project Guidelines 2026. Supplied assignment brief and marking guide. Presentation arrangements follow the later class message supplied by the student.

3. Flutter (n.d.) Flutter documentation. Available at: https://docs.flutter.dev/ (official reference for widgets, state and responsive user interfaces).

4. Supabase (n.d.) Auth documentation. Available at: https://supabase.com/docs/guides/auth.

5. Supabase (n.d.) Row Level Security. Available at: supabase.com/docs/guides/database/postgres/row-level-security.

6. Baseflow (n.d.) geolocator: Flutter geolocation package. Available at: https://pub.dev/packages/geolocator.

7. pdf and printing package contributors (n.d.) Dart PDF generation and Flutter printing/sharing. Available at: https://pub.dev/packages/pdf and https://pub.dev/packages/printing.

8. Republic of South Africa (1999) Rental Housing Act 50 of 1999. Available at: gov.za: Rental Housing Act 50 of 1999. Used for the distinction between rental security and automatic forfeiture; this project does not certify legal compliance.

9. Nkambule, M. (2026) BackyardPay source code, release 1.4.0, commit 87ce034. Available at: GitHub: BackyardPay release source. See docs/PROJECT-STRUCTURE.md, docs/design/USABILITY-REVIEW.md and docs/releases/1.4.0-simple-dashboards.md for structure, design rationale and checks.

Evidence provenance

The supplied screenshots show the deployed application with landlord and tenant accounts. Manual figures in this portfolio were captured from the same release using isolated demonstration accounts and a simulated backend; timestamps can reflect an advanced test clock. No real bank transfer, new local interview study or native Android test is claimed. The code excerpts are copied from the submitted repository, with file paths and source line numbers.